

Relatório Técnico — AgroSearch

Motor de Busca Inteligente para Textos Agrícolas

Equipe:	Caio Kleivson e Andre Luiz
Projeto:	AgroSearch — Desafio Integrador (Laboratório 04)
Disciplina:	Tópicos Avançados — Recuperação de Informação / PLN
Arquivo analisado:	Agrosearch.py

1. Objetivo e Metodologia

O programa **Agrosearch.py** implementa, em uma aplicação Streamlit, um motor de busca textual construído inteiramente do zero (sem bibliotecas de alto nível de PLN/ML, como scikit-learn ou NLTK). O sistema é organizado em três fases integradas: **(1)** pré-processamento de texto (tokenização, normalização, remoção de stopwords e stemming heurístico); **(2)** construção de um índice invertido (termo → documentos); e **(3)** busca e ranqueamento por TF-IDF, com um cálculo bônus de similaridade de cosseno. A base de testes é fixa e contém 5 documentos curtos sobre agricultura (irrigação, controle de pragas, adubação e cultivo).

Para este relatório, a lógica central do script foi executada de forma isolada da interface gráfica, com a configuração padrão do aplicativo (remoção de stopwords e stemming ativos), permitindo validar objetivamente os resultados produzidos em cada fase.

2. Resultados — Fase 1: Pré-processamento

A base contém 5 documentos, totalizando 66 tokens brutos. Após normalização (minúsculas e remoção de acentuação), o vocabulário possui 47 termos distintos; com stopwords e stemming aplicados, o vocabulário final foi reduzido para 33 termos — uma redução de 14 termos, evidenciando o efeito de compactação do pipeline.

Exemplo (Doc 1): "A soja requer irrigação constante durante o período de floração para garantir a produtividade."

Etapa	Resultado
1. Tokenização	A, soja, requer, irrigação, constante, durante, o, período, de, floração, para, garantir, a, produtividade
2. Normalização	a, soja, requer, irrigacao, constante, durante, o, periodo, de, floracao, para, garantir, a, produtividade
3. Sem stopwords	soja, requer, irrigacao, constante, periodo, floracao, garantir, produtividade
4. Stemming (final)	soja, requer, irrig, constante, periodo, flor, garantir, produtiv

Observa-se o comportamento esperado do stemmer heurístico: "irrigação" → "irrig" e "produtividade" → "produtiv", por corte de sufixos nominalizadores.

3. Resultados — Fase 2: Índice Invertido

O índice invertido construído a partir dos tokens finais possui 33 termos únicos, cada um associado à lista ordenada de documentos em que ocorre. A amostra abaixo confirma o funcionamento correto da estrutura, com termos discriminativos mapeados para documentos específicos (ex.: "biolog" aparece somente no Doc 2, relativo a controle biológico).

Termo	Documento(s)
adub	Doc 3
agua	Doc 5
algodao	Doc 4
biolog	Doc 2

causam	Doc 4
constante	Doc 1
controle	Doc 2
cultivo	Doc 5
cultura	Doc 4
desfolh	Doc 4
economiza	Doc 5
feito	Doc 2

(Amostra dos 12 primeiros de 33 termos indexados, em ordem alfabética.)

4. Resultados — Fase 3: Busca, TF-IDF e Similaridade de Cosseno

Para validar o motor de busca, foi executada a consulta **"irrigação soja"**, que após o pipeline de pré-processamento resultou nos termos de busca ['irrig', 'soja']. A tabela a seguir apresenta o score TF-IDF acumulado e a similaridade de cosseno de cada documento em relação à consulta.

Documento	TF-IDF Acumulado	Similaridade Cosseno
Doc 1	0.0775	0.2572
Doc 5	0.0568	0.1977
Doc 2	0.0317	0.0664
Doc 4	0.0277	0.0610
Doc 3	0.0000	0.0000

Resultado do ranqueamento: tanto por TF-IDF acumulado quanto por similaridade de cosseno, o documento mais relevante para a consulta foi o **Doc 1** ("A soja requer irrigação constante durante o período de floração para garantir a produtividade."), seguido do Doc 5, ambos contendo os termos "irrigação" e/ou "soja". O Doc 3, que trata de adubação e milho, obteve score zero em ambas as métricas por não conter nenhum termo da consulta, confirmando a coerência do ranqueamento com o conteúdo semântico dos documentos.

Os dois rankings coincidiram nesta consulta. Isso é esperado quando a consulta tem poucos termos e estes têm peso relevante nos documentos: o TF-IDF acumulado soma apenas os termos da query, enquanto o cosseno normaliza pelo vetor completo do documento — divergências tendem a surgir em consultas mais longas ou com termos de frequência muito distinta.

5. Conclusão

A execução do Agrosearch.py demonstrou que todas as três fases do pipeline — pré-processamento, indexação invertida e ranqueamento por TF-IDF/cosseno — foram implementadas corretamente e do zero, sem uso de bibliotecas prontas de PLN. Os resultados obtidos para a consulta de teste são consistentes com o conteúdo semântico da base de documentos, validando o funcionamento do motor de busca proposto para o estudo de caso da AgroTech Solutions.